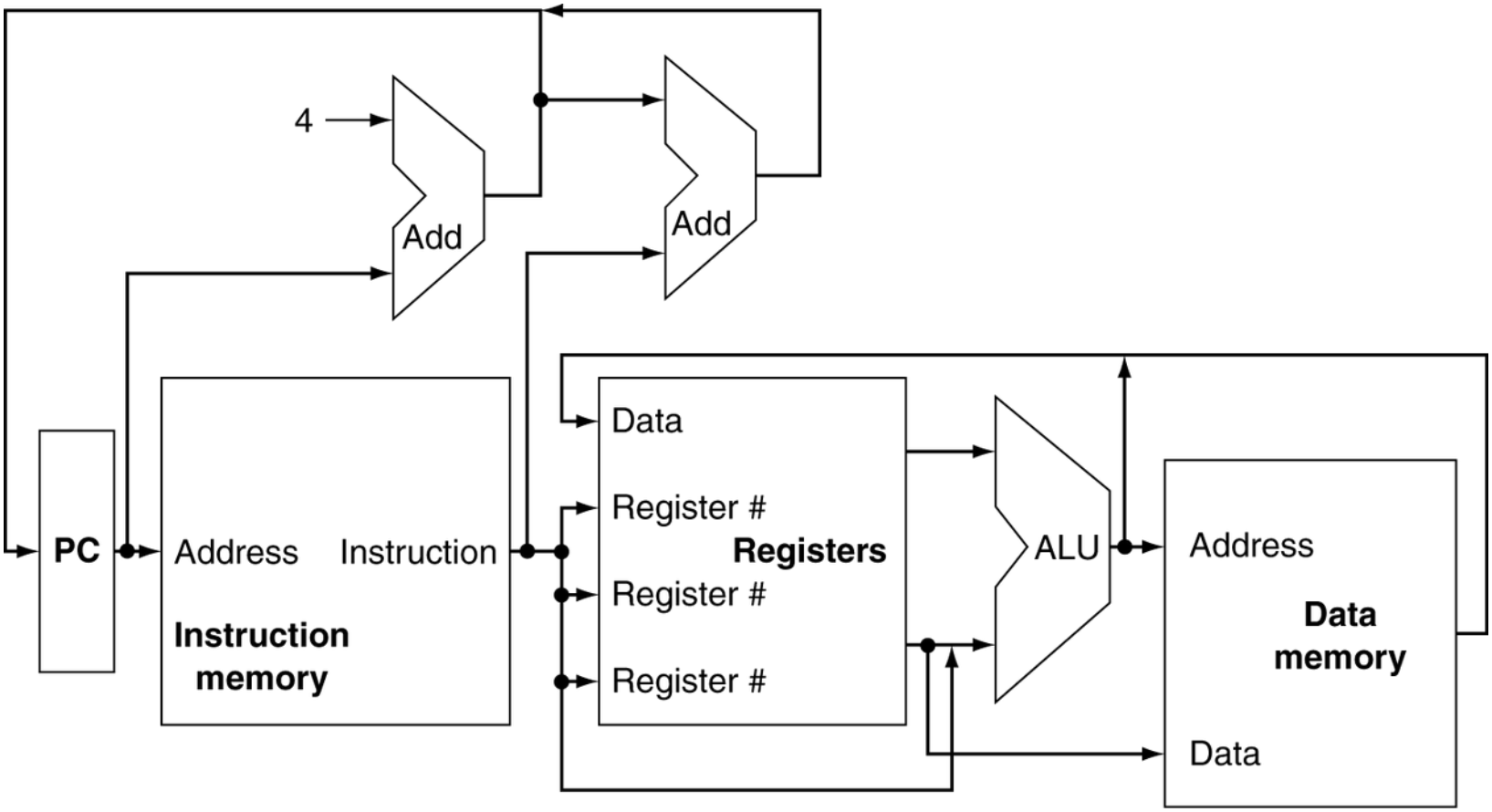
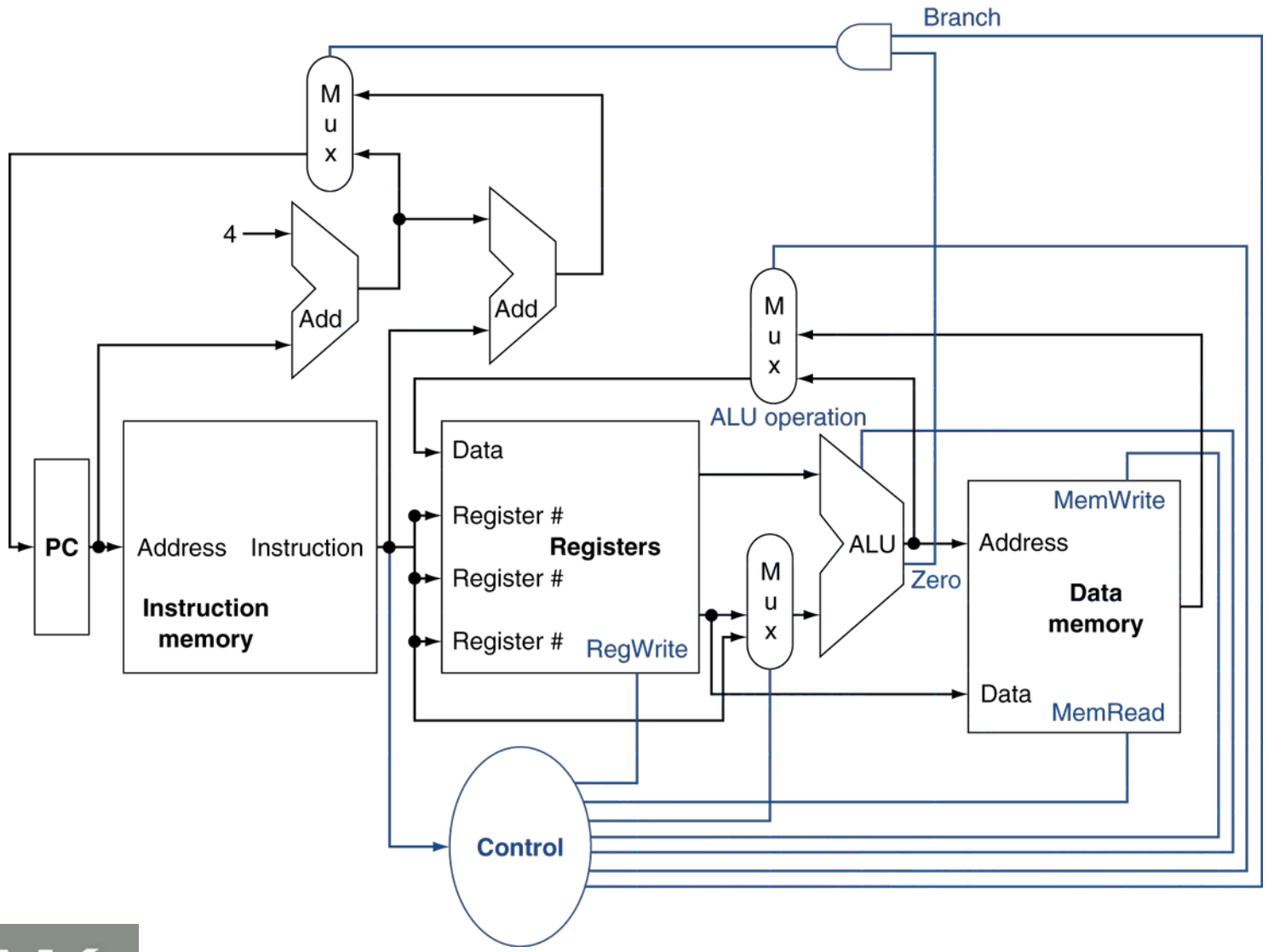
Chapter 3

The Processor

CPU Overview (Recap)



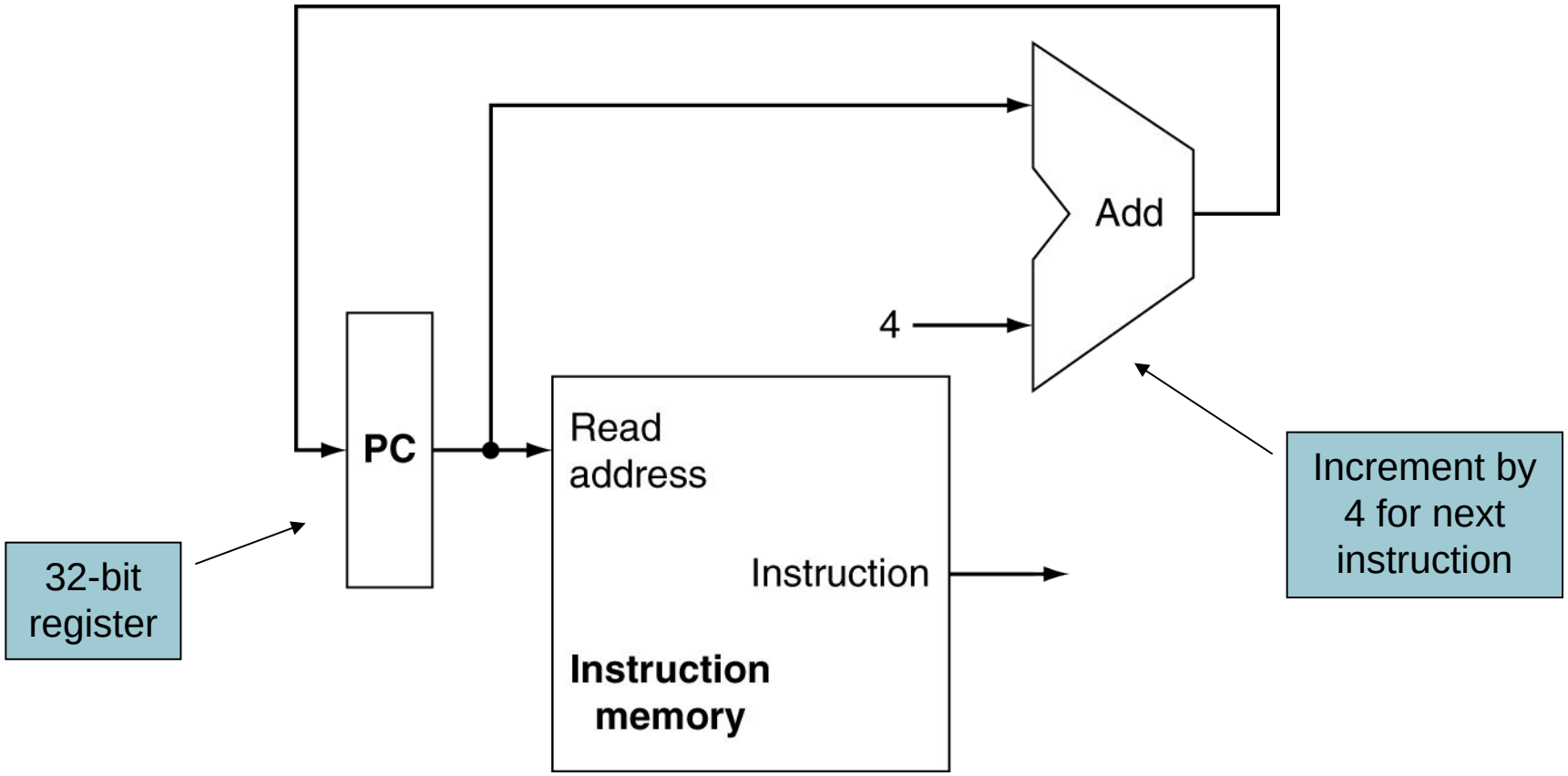
Control (Recap)



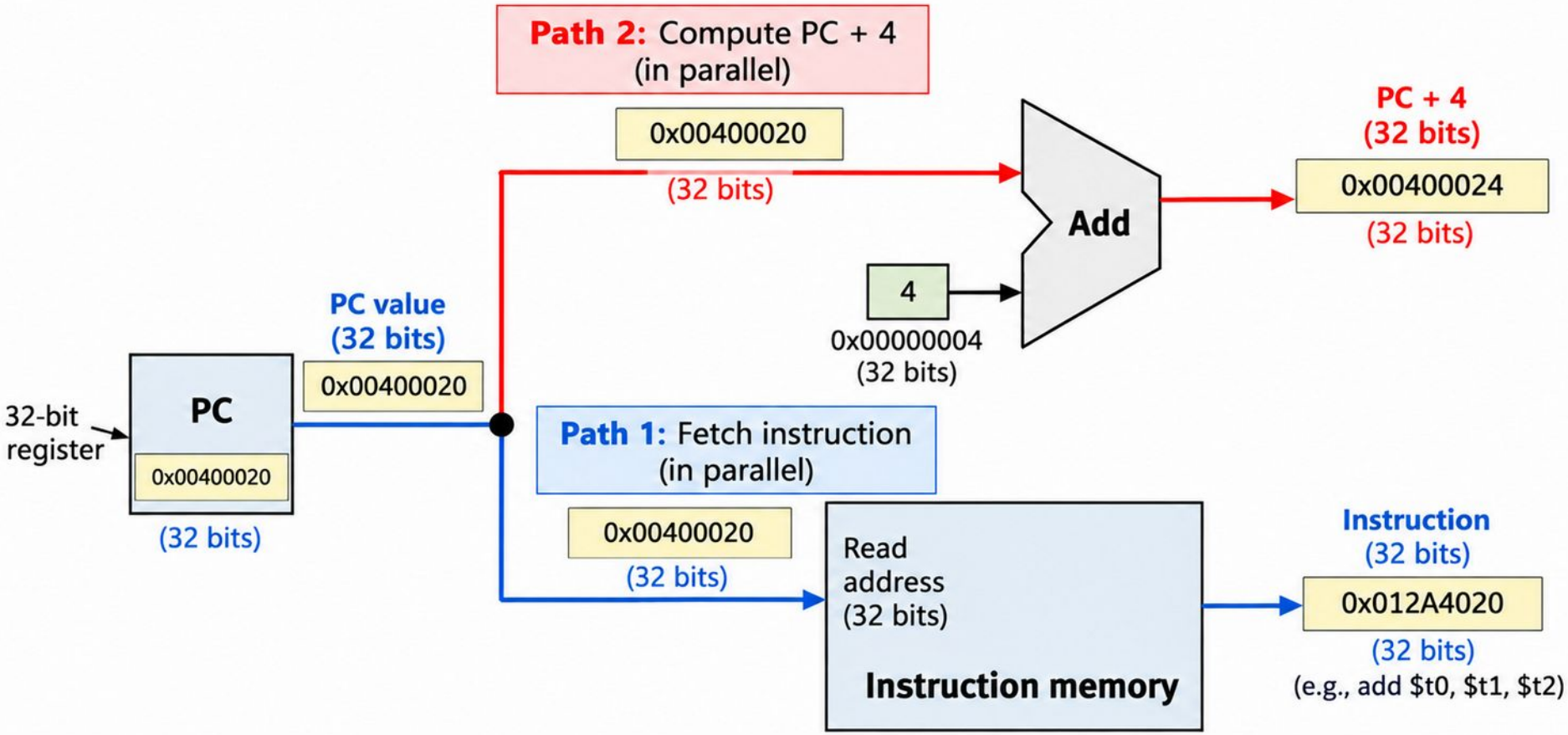
Building a Datapath

- Datapath
  - Elements that process data and addresses in the CPU
    - Registers, ALUs, mux's, memories, …
- We will build a MIPS datapath incrementally
  - Refining the overview design

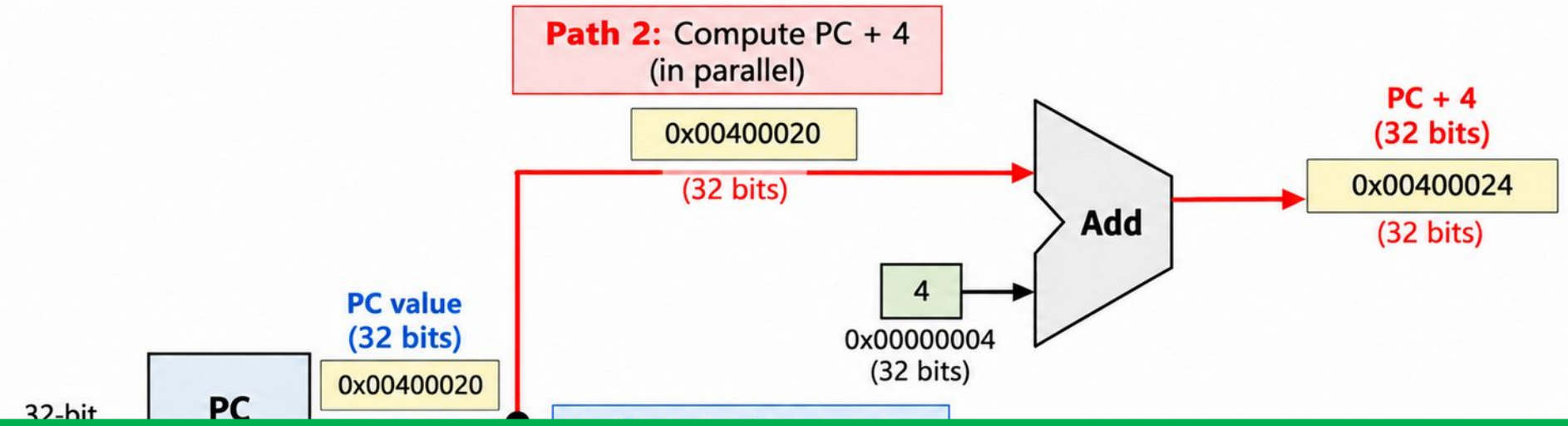
Instruction Fetch



Tracing the Instruction Fetch



Tracing the Instruction Fetch



Q1. Given PC = 0x00400020 and Instruction Memory[0x00400020] = 0x012A4020: What are the outputs produced during instruction fetch?

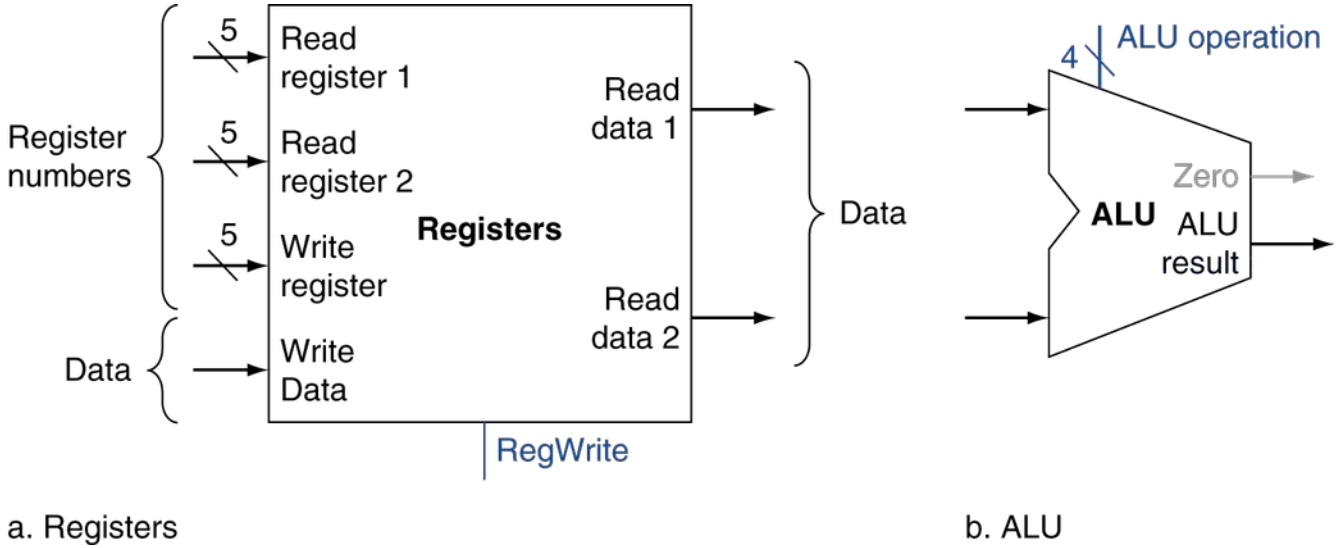
A1.

Instruction = 0x012A4020

PC + 4 = 0x00400024

R-Format Instructions

- Read two register operands
- Perform arithmetic/logical operation
- Write register result



Tracing R-Format Inst.

Example Instruction

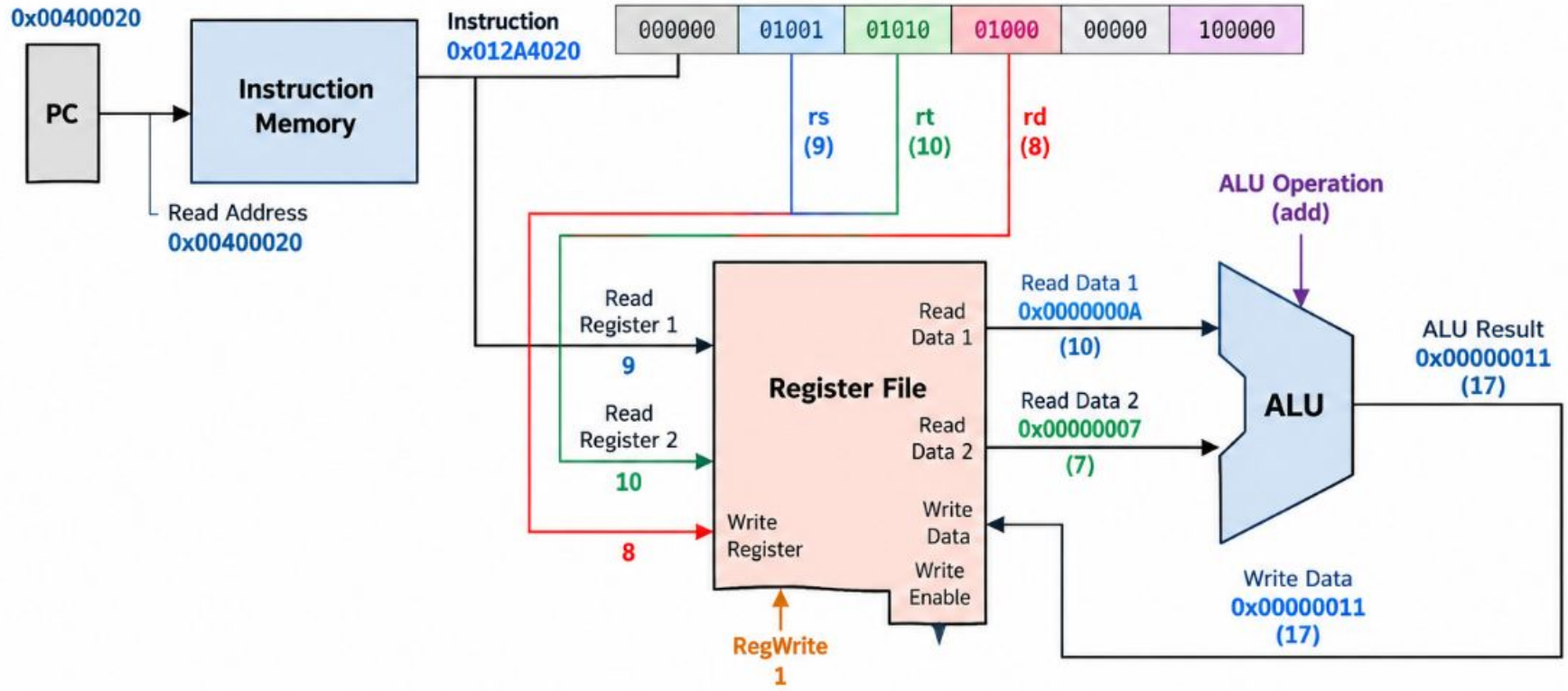
```
add $t0, $t1, $t2
$t0 = $t1 + $t2
```

Given Register Values

```
$t1 ($9) = 10 (0x0000000A)
$t2 ($10) = 7  (0x00000007)
```

Instruction Encoding (R-format)

opcode	rs	rt	rd	shamt	funct
000000	01001	01010	01000	00000	100000
0x00	$9 ($t1)	$10 ($t2)	$8 ($t0)	0	0x20 (add)



Tracing R-Format Inst.

Example Instruction

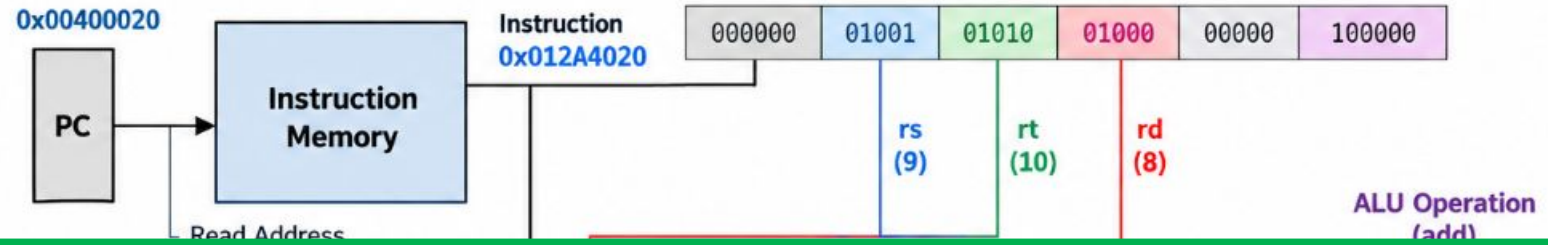
```
add $t0, $t1, $t2
$t0 = $t1 + $t2
```

Given Register Values

```
$t1 ($9) = 10 (0x0000000A)
$t2 ($10) = 7  (0x00000007)
```

Instruction Encoding (R-format)

opcode	rs	rt	rd	shamt	funct
000000	01001	01010	01000	00000	100000
0x00	$9 ($t1)	$10 ($t2)	$8 ($t0)	0	0x20 (add)



Q2. During an R-format instruction:
rs = 9, rt = 10, rd = 8
Read Data 1 = 15, Read Data 2 = 6, ALU Result = 21
Which instruction is being executed, and what value is written back?

A2.
add $t0, $t1, $t2
$t0 <- 21

Values in R-Format Inst.

Value	Width	Example	Datapath Role
Instruction	32 bits	0x012A4020	Fetched R-format instruction
rs	5 bits	9 ($t1)	Selects Read Register 1
rt	5 bits	10 ($t2)	Selects Read Register 2
rd	5 bits	8 ($t0)	Selects the destination register
Read Data 1	32 bits	10	Value read from $t1
Read Data 2	32 bits	7	Value read from $t2
ALU Result	32 bits	17	Result produced by the ALU
Write Register	5 bits	8 ($t0)	Register selected for write-back
Write Data	32 bits	17	Value returned to the Register File

Number vs. Data

Register Number
(5-bit)

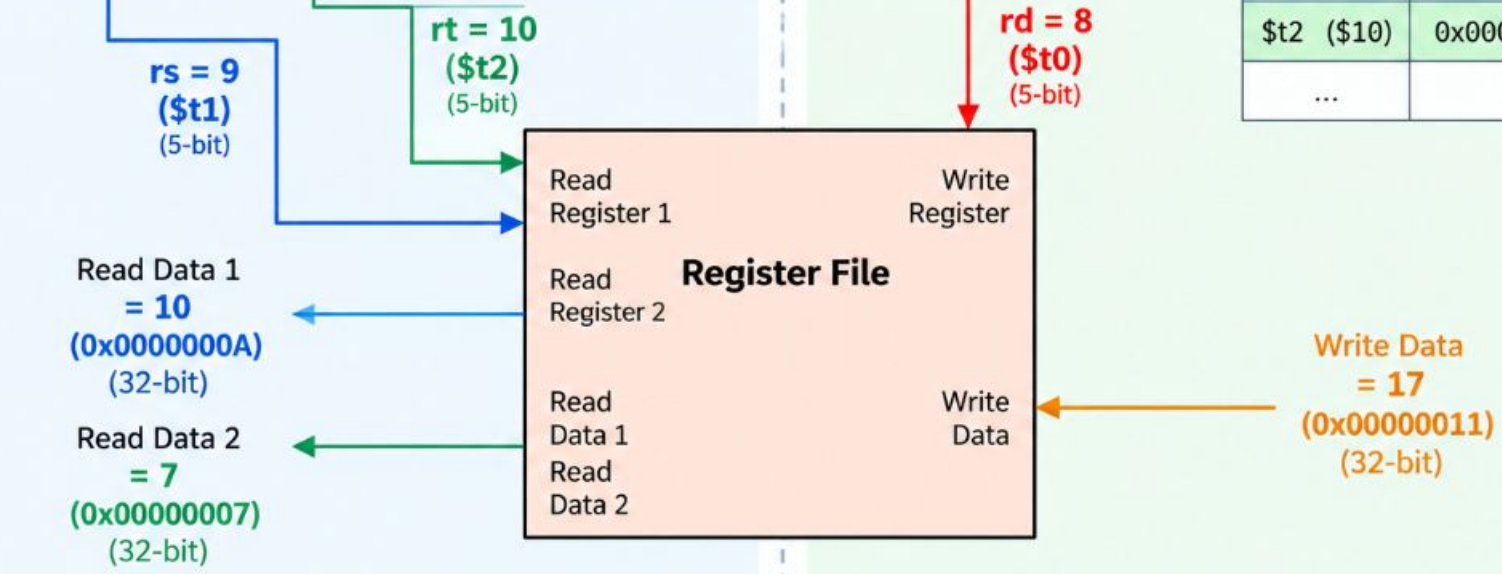
- Specifies **which register** to access
- Used as an **index** to the register file



Register Data
(32-bit)

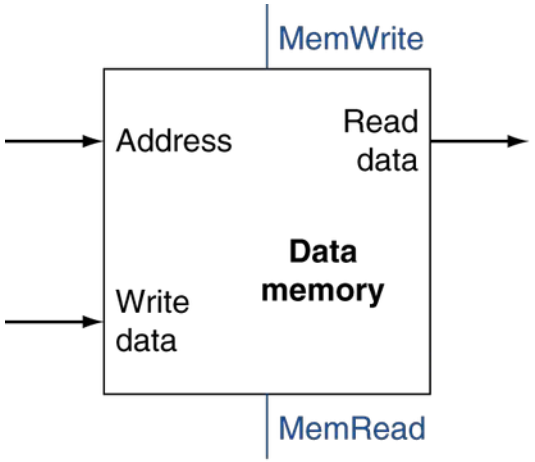
- The **actual 32-bit value** stored in the register
- Read from the register file **using the register number**

Register	Value (32-bit)
$t0 ($8)	...
$t1 ($9)	0x0000000A (10)
$t2 ($10)	0x00000007 (7)
...	...

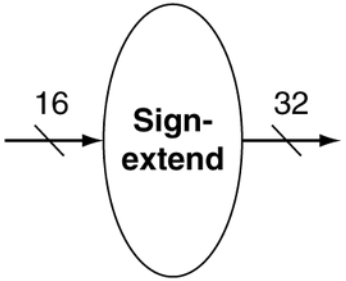


Load/Store Instructions

- Read register operands
- Calculate address using 16-bit offset
  - Use ALU, but sign-extend offset
- Load: Read memory and update register
- Store: Write register value to memory



a. Data memory unit



b. Sign extension unit

Tracing Load Instruction

Example Instruction

```
lw  $t0, 12($t1)
$t0 = Mem[$t1 + 12]
```

Given Values

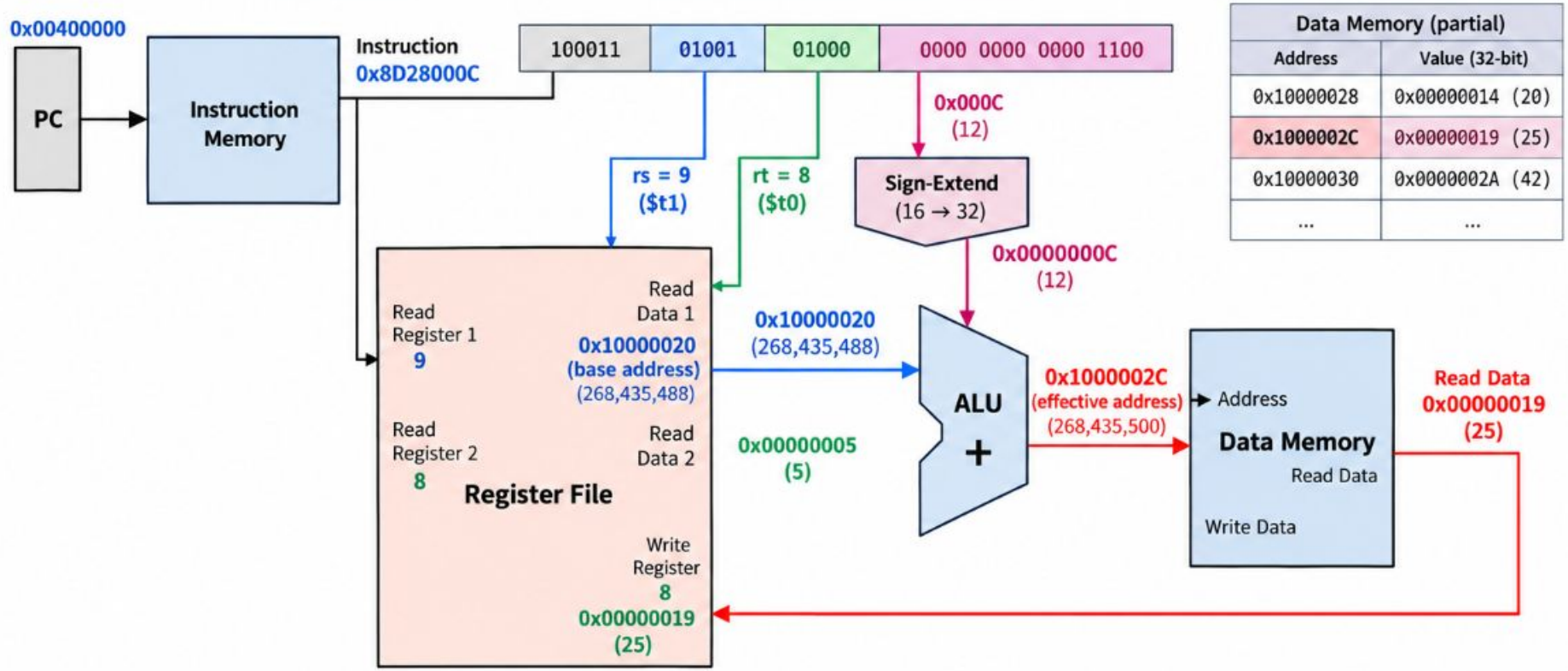
$t1 ($9) = 0x10000020 (base address)

Memory[0x1000002C] = 25 (0x00000019)

Other registers are not used.

Instruction Encoding (I-format)

opcode	rs	rt	immediate (16-bit)
100011	01001	01000	0000 0000 0000 1100
0x23	$9 ($t1)	$8 ($t0)	0x000C (12)



Data Memory (partial)

Address	Value (32-bit)
0x10000028	0x00000014 (20)
0x1000002C	0x00000019 (25)
0x10000030	0x0000002A (42)
...	...

Tracing Store Instruction

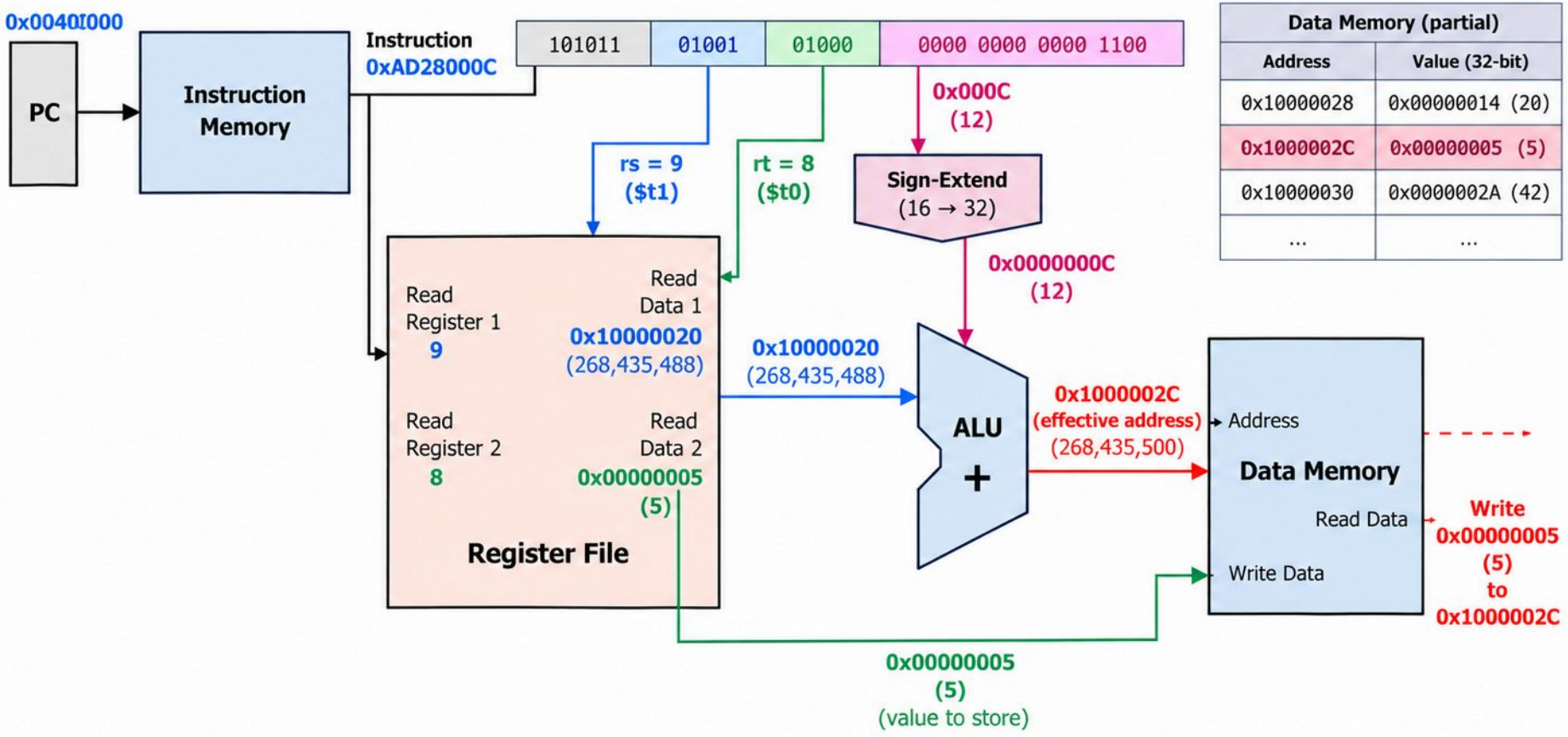
```
sw  $t0, 12($t1)
Mem[$t1 + 12] = $t0
```

$t1 ($9) = 0x10000020 (base address)

$t0 ($8) = 0x00000005 (value to store)

Memory[0x1000002C] will be updated to 5.

opcode	rs	rt	immediate (16-bit)
101011	01001	01000	0000 0000 0000 1100
0x2B	**$9** ($t1)	**$8** ($t0)	0x000C (12)



Data Memory (partial)	
Address	**Value (32-bit)**
0x10000028	0x00000014 (20)
0x1000002C	0x00000005 (5)
0x10000030	0x0000002A (42)
...	...

Values in Load/Store Inst.

Value	Width	lw $t0, 12($t1)	sw $t0, 12($t1)	Datapath Role
Instruction	32 bits	0x8D28000C	0xAD28000C	Complete machine instruction
rs	5 bits	9 ($t1)	9 ($t1)	Selects the base register
rt	5 bits	8 ($t0)	8 ($t0)	Destination for lw; source data register for sw
Immediate	16 bits	12	12	Address offset
Read Data 1	32 bits	0x10000020	0x10000020	Base address from $t1
Read Data 2	32 bits	—	0x00000005	Value from $t0 to be stored
Sign-Extended Immediate	32 bits	0x0000000C	0x0000000C	32-bit offset sent to ALU
ALU Result	32 bits	0x1000002C	0x1000002C	Effective memory address
Memory Read Data	32 bits	0x00000019	—	Value loaded from memory
Memory Write Data	32 bits	—	0x00000005	Value written into memory
Write Register	5 bits	8 ($t0)	—	Destination register for lw
Write Data	32 bits	0x00000019	—	Value written back to $t0

Three Highlights

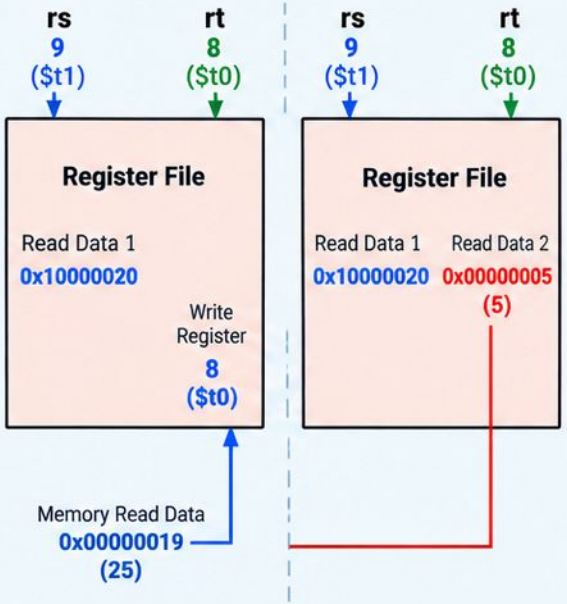
1 Different role of rt

`lw $t0, 12($t1)`

rt = 8 ($t0)
destination register

`sw $t0, 12($t1)`

rt = 8 ($t0)
source register



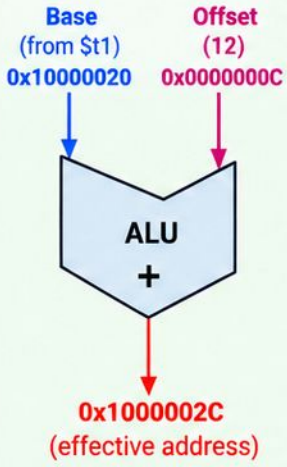
- For **lw**: rt specifies the **destination register** (write-back).
- For **sw**: rt specifies the **source register** (the value to be stored).

2 ALU Result is the Effective Memory Address

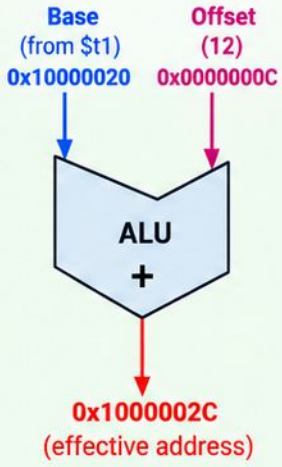
Effective Address = Base Register + Sign-Extended Offset

ALU Result = Memory Address
(not the data to be loaded/stored)

`lw $t0, 12($t1)`



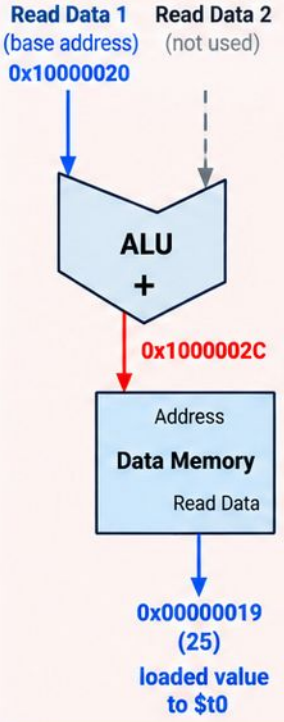
`sw $t0, 12($t1)`



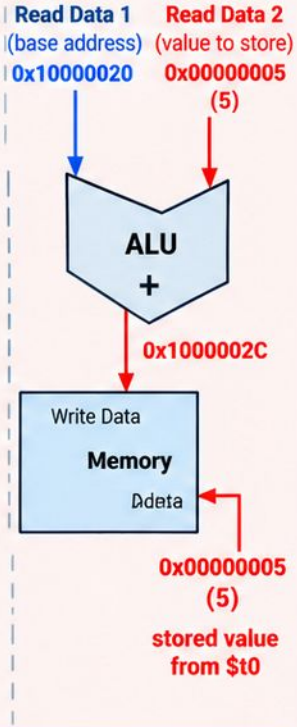
- The ALU does not compute the data value.
- It computes the **effective memory address** (Base + Offset) for both lw and sw.
- The same address calculation hardware is used by both instructions.

3 Different use of Read Data 2

`lw $t0, 12($t1)`

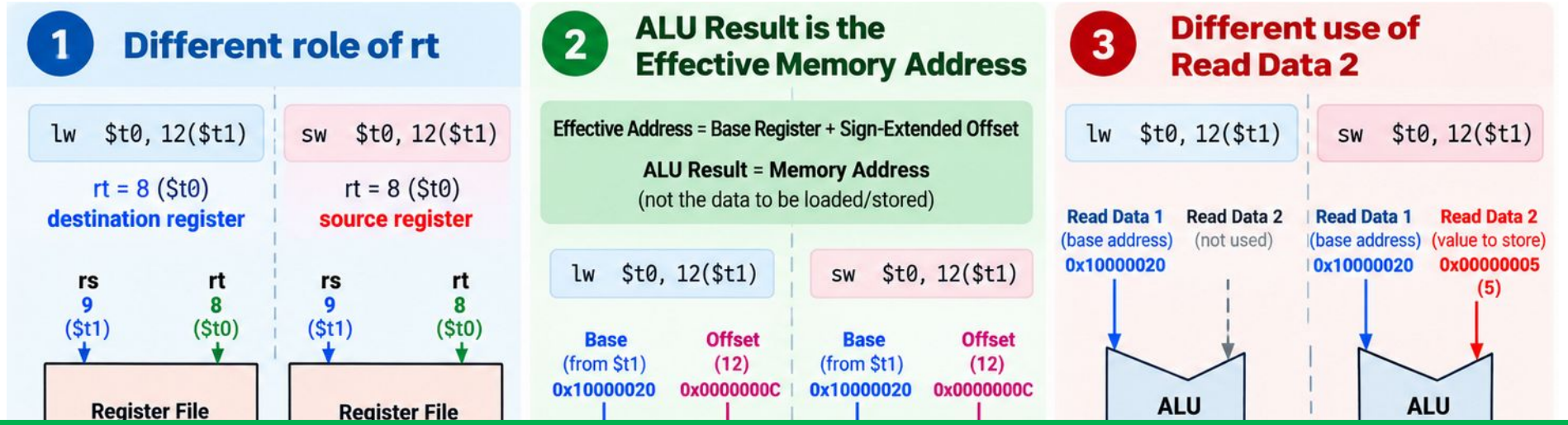


`sw $t0, 12($t1)`



- For **lw**: Read Data 2 is **not used**.
- For **sw**: Read Data 2 provides the **value to be written into memory**.

Three Highlights



Q3. After address calculation, the datapath has:
ALU Result = 0x1000002C
Read Data 2 = 25
No data is written back to the Register File
Is this lw or sw? What happens to the value 25?

A3.

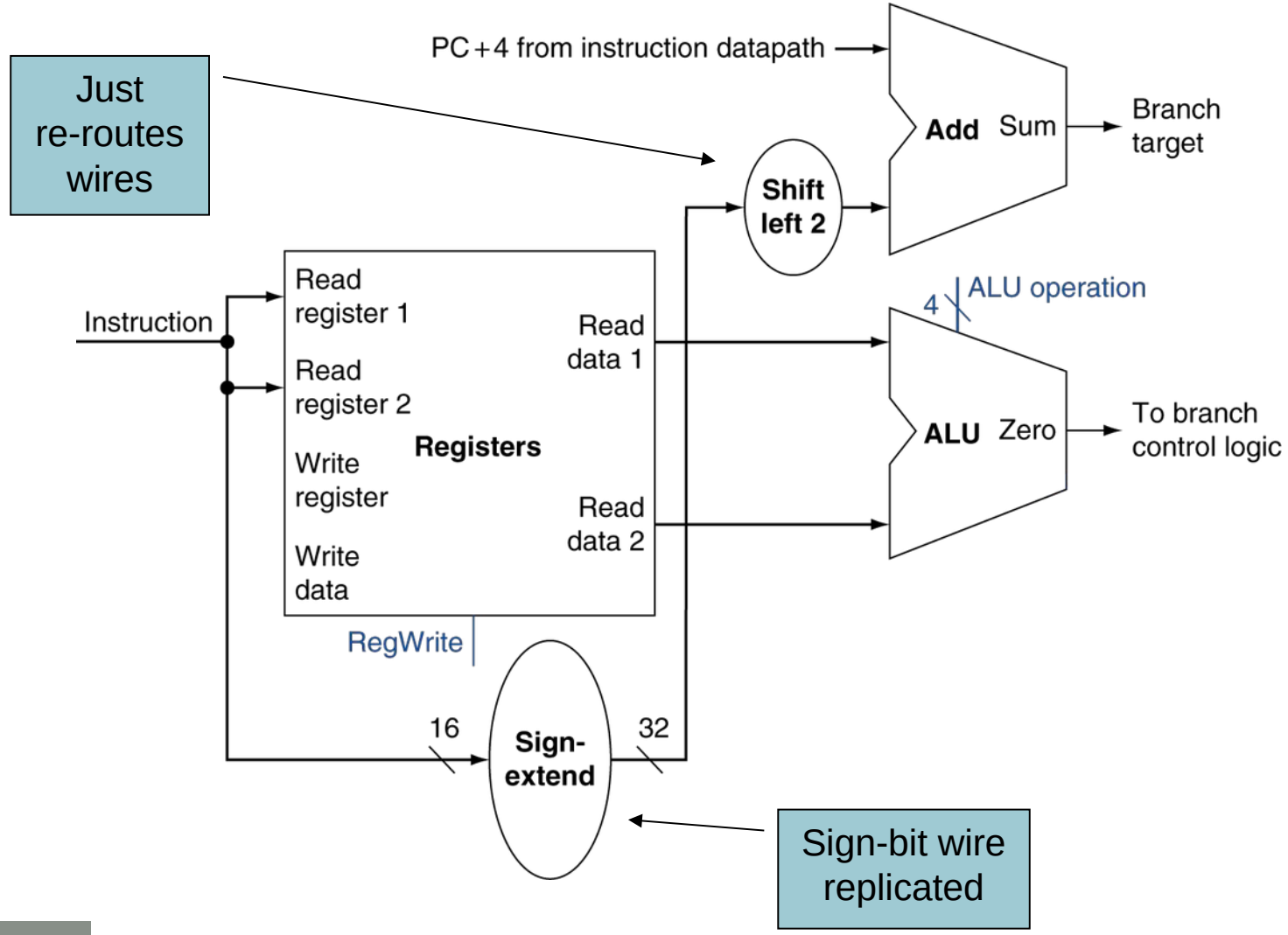
sw.

The value 25 is written to memory address 0x1000002C.

Branch Instructions

- Read register operands
- Compare operands
  - Use ALU, subtract and check Zero output
- Calculate target address
  - Sign-extend displacement
  - Shift left 2 places (word displacement)
  - Add to PC + 4
    - Already calculated by instruction fetch

Branch Instructions



Tracing Branch Instructions

```
beq  $t0, $t1, 3
```

if ($t0 == $t1) PC ← PC + 4 + (3 x 4)

Given Values

PC = 0x00400020

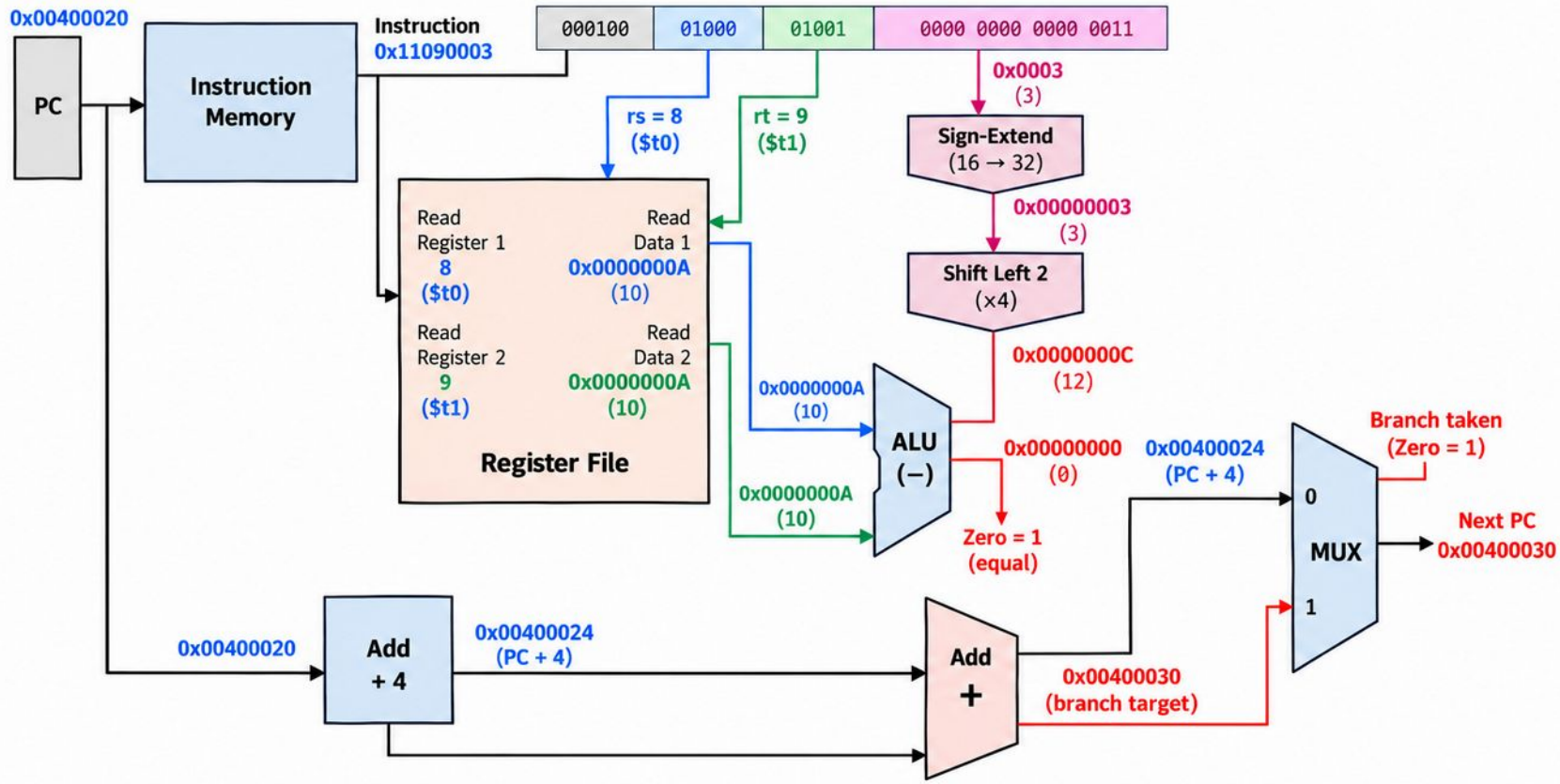
$t0 ($8) = 10 (0x0000000A)

$t1 ($9) = 10 (0x0000000A)

Other registers are not used.

Instruction Encoding (I-format)

opcode	rs	rt	immediate (16-bit)
000100	01000	01001	0000 0000 0000 0011
0x04	8 ($t0)	9 ($t1)	0x0003 (3)



Values in Branch Instructions

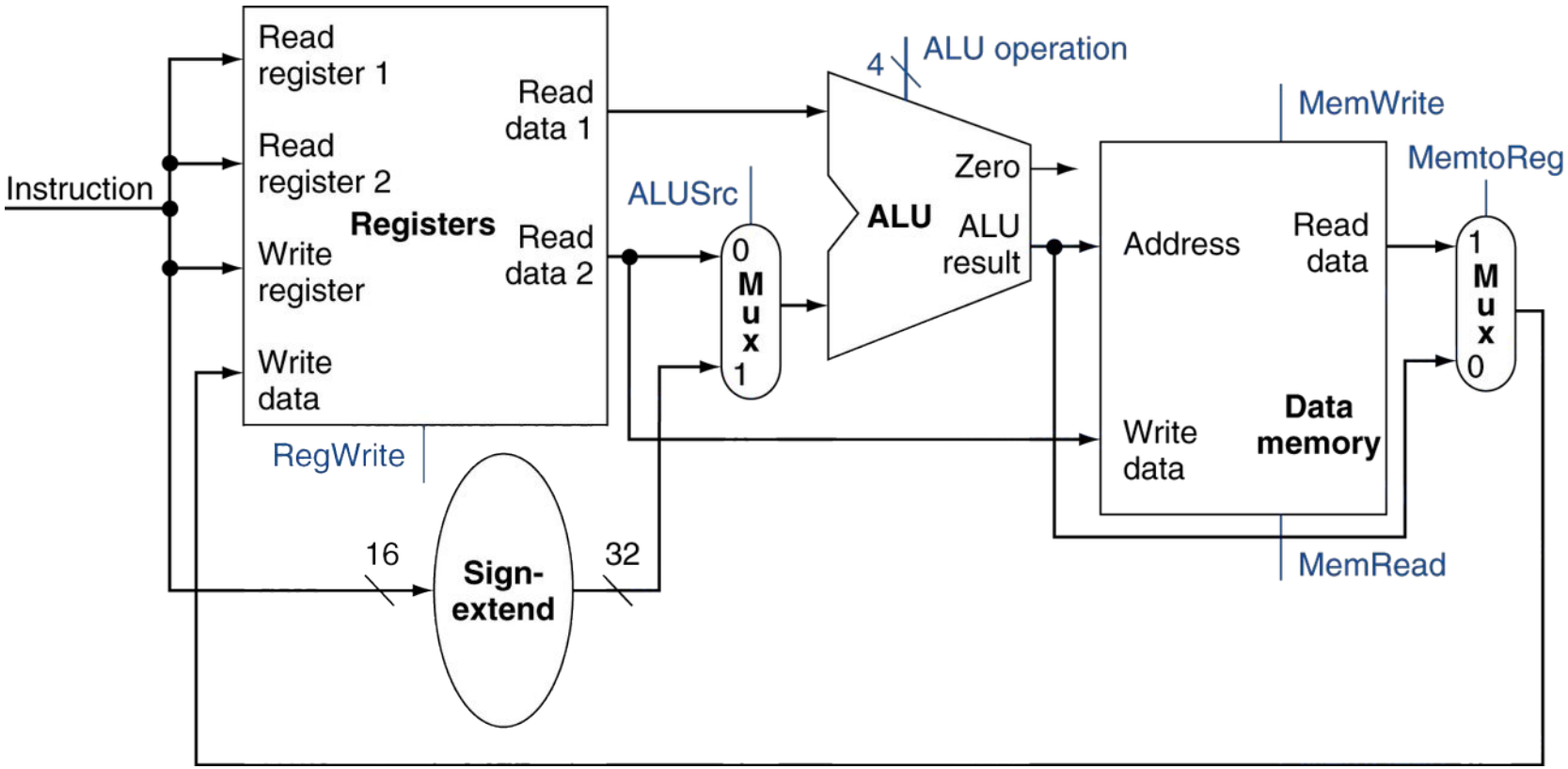
Value	Width	Example	Datapath Role
PC	32 bits	0x00400020	Address of the current branch instruction
PC + 4	32 bits	0x00400024	Base address for branch target calculation
rs	5 bits	8 ($t0)	Selects Read Register 1
rt	5 bits	9 ($t1)	Selects Read Register 2
Read Data 1	32 bits	10	First value for comparison
Read Data 2	32 bits	10	Second value for comparison
Immediate	16 bits	3	Signed branch displacement
Sign-Extended Immediate	32 bits	0x00000003	32-bit signed displacement
Shifted Offset	32 bits	0x0000000C	Byte offset after << 2
ALU Result	32 bits	0	Result of 10 − 10
Zero	1 bit	1	Indicates the two register values are equal
Branch Target	32 bits	0x00400030	PC + 4 + (offset << 2)
Next PC	32 bits	0x00400030	Address selected for the next instruction

Values in Branch Instructions

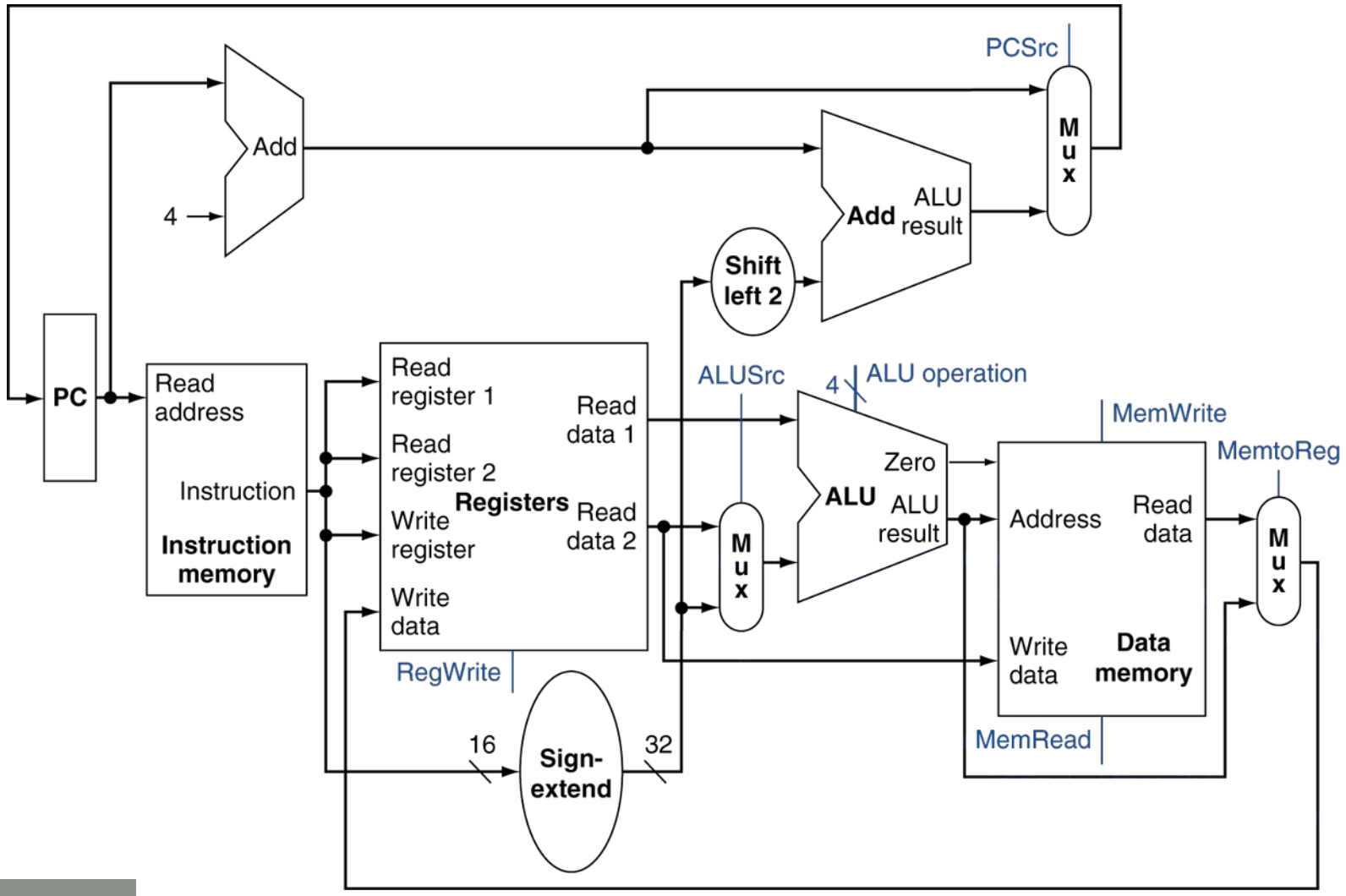
Value	Width	Example	Datapath Role
PC	32 bits	0x00400020	Address of the current branch instruction
PC + 4	32 bits	0x00400024	Base address for branch target calculation
Next PC	32 bits	0x00400030	Address selected for the next instruction

Q4. Given:
PC = 0x00400040rs = 8 ($t0), rt = 9 ($t1)
Read Data 1 = 15, Read Data 2 = 15, Shifted Offset = 16
(a)What branch instruction is being executed?
(b) What are ALU Result and Zero?
(c) What are Branch Target and Next PC?
A4.
(a) beq $t0, $t1, 4
(b) ALU Result = 0, Zero = 1
(c) Branch Target = 0x00400054, Next PC = 0x00400054

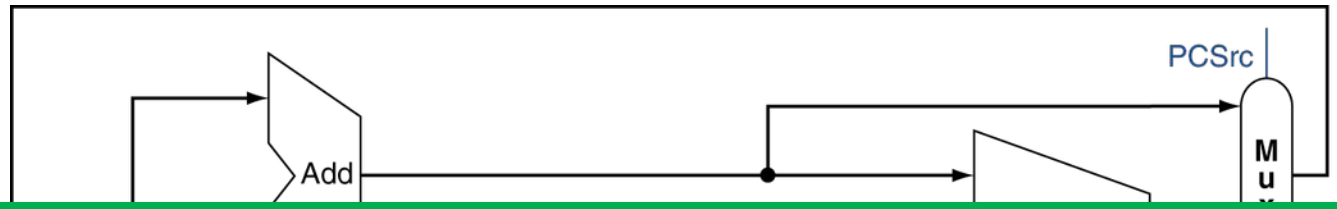
R-Type/Load/Store Datapath



Full Datapath



Full Datapath



Q5. During execution:
Register File reads two registers.
ALU calculates an address using a sign-extended immediate.
Data Memory is accessed.
Memory provides a value.
The value is written back to the Register File.
(a) What type of instruction is being executed?
(b) If rs = 9 ($t1), rt = 8 ($t0), and Immediate = 12, what is the instruction?

A5.

(a)Load (lw)

(b) lw $t0, 12($t1)

Section 3-3 Quick Questions

Q1. Given PC = 0x00400020 and Instruction Memory[0x00400020] = 0x012A4020, what are the two outputs produced during instruction fetch?

Q2. Given rs = 9 ($t1), rt = 10 ($t2), rd = 8 ($t0), Read Data 1 = 15, Read Data 2 = 6, and ALU Result = 21, which R-format instruction is being executed?

Q3. ALU Result = 0x1000002C, Read Data 2 = 25, and no data is written back to the Register File. Is this lw or sw, and what happens to 25?

Q4. Given PC = 0x00400040, rs = 8 ($t0), rt = 9 ($t1), Read Data 1 = 15, Read Data 2 = 15, and Shifted Offset = 16, identify the branch instruction and determine the Branch Target and Next PC.

Q5. The ALU uses a sign-extended immediate, Data Memory is read, and the memory value is written back to the Register File. What type of instruction is being executed?

Exercise 1 — Comparing Datapath Usage

Consider the following instructions:

add $t0, $t1, $t2 lw $t0, 8($t1) sw $t0, 8($t1) beq $t0, $t1, 2

For each instruction, determine whether it uses the five datapath components below.

Answer

Instruction	ALU	Sign Extend	Data Memory	Register Write	Branch Target Adder
add	✓	—	—	✓	—
lw	✓	✓	Read	✓	—
sw	✓	✓	Write	—	—
beq	✓	✓	—	—	✓

ALU roles: arithmetic (add), effective address (lw/sw), and comparison (beq).

Different instructions reuse the same datapath components for different purposes.

Exercise 2 — Debug the Datapath

For lw $t0, -8($t1):

$t1 = 0x10000020 Memory[0x10000018] = 25

A student traces the datapath as follows:

Immediate = 0xFFF8

Sign-Extended Immediate = 0x0000FFF8

Write Data = 0x10010018

ALU Result = 0x10010018

Find the errors and correct them.

Answer

There are three errors:

1. Sign-Extended Immediate = 0xFFFFFFF8
2. ALU Result = 0x10000018
3. Write Data = 25

The ALU Result is the memory address. For lw, the value read from memory is written back to $t0.

Trace the data flow in order:
sign extension → address calculation → memory read → write-back.

Exercise 3 — Complete Branch Tracing

Given:

PC = 0x00400080 $t0 = 20 $t1 = 20

Instruction: beq $t0, $t1, -4

(a) PC + 4 (b) ALU Result and Zero

(c) Sign-Extended Immediate and Shifted Offset

(d) Branch Target and Next PC

Answer

(a) PC + 4 = 0x00400084

(b) ALU Result = 0, Zero = 1

(c) Sign-Extended Immediate = 0xFFFFFFFC

(d) Branch Target = 0x00400074, Next PC = 0x00400074

Shifted Offset = 0xFFFFFFF0 (-16)

Branch tracing combines comparison and target-address calculation to determine the Next PC.